

Domain 5: Networking and Content Delivery

AWS Certified SysOps Administrator – Associate (SOA-C03)

Exam Weight: ~18% of scored content

Source: AWS Certified SysOps Administrator – Associate Exam Guide

Overview

Domain 5 covers the skills required to design, configure, optimize, and troubleshoot networking and content delivery on AWS. A CloudOps engineer must understand how to build and secure VPCs, connect workloads privately, protect network infrastructure, configure DNS and routing policies, accelerate content delivery, and diagnose connectivity problems using logs and monitoring tools.

Networking on AWS is built on a software-defined model — every component (subnets, route tables, security groups, gateways) is configurable via API, CLI, or console, and changes take effect immediately without hardware provisioning.

Task 5.1: Implement and Optimize Networking Features and Connectivity

Skill 5.1.1 – Configure a VPC

What is Amazon VPC?

Amazon Virtual Private Cloud (Amazon VPC) lets you provision a logically isolated section of the AWS Cloud where you can launch AWS resources in a virtual network that you define. You have complete control over your virtual networking environment, including selection of your own IP address range, creation of subnets, and configuration of route tables and network gateways.

Source: What is Amazon VPC?

Subnets

A **subnet** is a range of IP addresses in your VPC. You launch AWS resources (such as EC2 instances) into subnets. Each subnet must reside entirely within one Availability Zone.

Source: Subnets for your VPC

Subnet types:

Type	Description
Public subnet	Has a route to an internet gateway; resources can have public IP addresses and communicate with the internet
Private subnet	No direct route to an internet gateway; resources communicate with the internet only through a NAT gateway or NAT instance
Isolated subnet	No route to the internet at all; used for databases or internal services that should never reach the internet

Key subnet concepts:

- AWS reserves 5 IP addresses in each subnet (first 4 and last 1). A `/24` subnet gives you 251 usable addresses.
- Subnets in the same VPC can communicate with each other by default (controlled by security groups and NACLs).
- **Auto-assign public IPv4** can be enabled per subnet so that instances launched into it automatically receive a public IP.
- For IPv6, subnets can be assigned a `/64` prefix from the VPC's `/56` IPv6 CIDR block.

```
# Create a public subnet
aws ec2 create-subnet \
  --vpc-id vpc-0abc12345 \
  --cidr-block 10.0.1.0/24 \
  --availability-zone us-east-1a

# Enable auto-assign public IPv4 for the subnet
aws ec2 modify-subnet-attribute \
  --subnet-id subnet-0abc12345 \
  --map-public-ip-on-launch
```

Route Tables

A **route table** contains a set of rules (routes) that determine where network traffic from your subnet or gateway is directed. Every subnet must be associated with a route table.

Source: Configure route tables

Route table types:

Type	Description
Main route table	Automatically created with the VPC; used by subnets not explicitly associated with another route table
Custom route table	Created by you; explicitly associated with one or more subnets
Gateway route table	Associated with an internet gateway or virtual private gateway for gateway routing

Common routes:

Destination	Target	Purpose
10.0.0.0/16	local	Local VPC traffic (always present, cannot be deleted)
0.0.0.0/0	igw-xxxxxxx	Default route to internet gateway (public subnet)
0.0.0.0/0	nat-xxxxxxx	Default route to NAT gateway (private subnet)
10.1.0.0/16	tgw-xxxxxxx	Route to another VPC via Transit Gateway
0.0.0.0/0	vgw-xxxxxxx	Route to on-premises via Virtual Private Gateway

Longest prefix match is used when multiple routes match — the most specific route wins.

Network Access Control Lists (NACLs)

A **network ACL (NACL)** is a stateless, optional layer of security for your VPC that acts as a firewall for controlling traffic in and out of one or more subnets.

Source: Control subnet traffic with network access control lists

Key NACL characteristics:

Characteristic	NACL	Security Group
Scope	Subnet level	Instance/ENI level
Statefulness	Stateless — return traffic must be explicitly allowed	Stateful — return traffic is automatically allowed
Rule evaluation	Rules evaluated in order (lowest number first); first match wins	All rules evaluated; most permissive wins
Default behavior	Default NACL allows all traffic; custom NACLs deny all by default	Default SG allows all outbound, denies all inbound
Allow/Deny	Supports both Allow and Deny rules	Allow rules only

Ephemeral ports: Because NACLs are stateless, you must allow inbound ephemeral ports (1024–65535) on private subnets to permit return traffic from internet-bound requests.

```
# Example NACL rules for a public subnet

Rule 100: ALLOW TCP 0.0.0.0/0 port 443 (HTTPS inbound)

Rule 110: ALLOW TCP 0.0.0.0/0 port 80 (HTTP inbound)

Rule 120: ALLOW TCP 0.0.0.0/0 ports 1024-65535 (ephemeral return traffic)

Rule *:    DENY all (implicit deny)
```

Security Groups

A **security group** acts as a virtual firewall for EC2 instances and other resources, controlling inbound and outbound traffic at the instance level.

Key security group characteristics:

- **Stateful** — if you allow inbound traffic on port 443, the response is automatically allowed outbound.
- Rules are **allow-only** — you cannot create deny rules.
- Multiple security groups can be attached to a single instance (up to 5 by default).
- Security groups can reference other security groups as sources/destinations (useful for tiered architectures).
- Changes take effect immediately.

Best practices:

- Use the **principle of least privilege** — only open ports that are required.
- Reference security group IDs rather than IP ranges for inter-tier communication (e.g., allow the web tier SG to reach the app tier SG on port 8080).
- Avoid `0.0.0.0/0` on sensitive ports (SSH port 22, RDP port 3389).
- Use **AWS Systems Manager Session Manager** instead of opening SSH/RDP ports.

```
# Create a security group
aws ec2 create-security-group \
  --group-name web-sg \
  --description "Web tier security group" \
  --vpc-id vpc-0abc12345

# Allow HTTPS inbound from anywhere
aws ec2 authorize-security-group-ingress \
  --group-id sg-0abc12345 \
  --protocol tcp \
  --port 443 \
  --cidr 0.0.0.0/0
```

Internet Gateway

An **internet gateway (IGW)** is a horizontally scaled, redundant, and highly available VPC component that allows communication between your VPC and the internet. It serves two purposes: providing a target in your VPC route tables for internet-routable traffic, and performing network address translation (NAT) for instances with public IPv4 addresses.

Source: Enable internet access for a VPC using an internet gateway

Requirements for internet access via IGW:

- 1. An internet gateway attached to the VPC.
- 2. A route in the subnet's route table pointing `0.0.0.0/0` to the IGW.
- 3. The instance must have a public IPv4 address or Elastic IP address.
- 4. Security groups and NACLs must allow the relevant traffic.

```
# Create and attach an internet gateway
aws ec2 create-internet-gateway
aws ec2 attach-internet-gateway \
  --internet-gateway-id igw-0abc12345 \
  --vpc-id vpc-0abc12345

# Add a route to the public subnet's route table
aws ec2 create-route \
  --route-table-id rtb-0abc12345 \
  --destination-cidr-block 0.0.0.0/0 \
  --gateway-id igw-0abc12345
```

NAT Gateway

A **NAT gateway** enables instances in a private subnet to initiate outbound connections to the internet (or other AWS services) while preventing the internet from initiating connections to those instances.

Source: NAT gateways

NAT gateway types:

Type	Description
Public NAT gateway	Deployed in a public subnet; uses an Elastic IP; allows private instances to reach the internet
Private NAT gateway	Deployed in a private subnet; used for private connectivity between VPCs or on-premises networks without internet access

Key characteristics:

- Managed by AWS — no patching, no failover configuration required.
- Scales automatically up to 100 Gbps.
- Supports up to 55,000 simultaneous connections per Elastic IP address.
- **One NAT gateway per AZ** is recommended for high availability (avoid cross-AZ NAT traffic charges).
- NAT gateways do not support security groups — use NACLs for subnet-level control.

```
# Create a public NAT gateway (requires an Elastic IP)
aws ec2 allocate-address --domain vpc
aws ec2 create-nat-gateway \
  --subnet-id subnet-public-0abc12345 \
  --allocation-id eipalloc-0abc12345

# Add a route in the private subnet's route table
aws ec2 create-route \
  --route-table-id rtb-private-0abc12345 \
  --destination-cidr-block 0.0.0.0/0 \
  --nat-gateway-id nat-0abc12345
```

Egress-Only Internet Gateway

An **egress-only internet gateway** is a VPC component that allows outbound communication over IPv6 from instances in your VPC to the internet, while preventing the internet from initiating IPv6 connections to your instances. It is the IPv6 equivalent of a NAT gateway.

Key points:

- Only works with IPv6 traffic.
- Stateful — return traffic is automatically allowed.
- No Elastic IP required (IPv6 addresses are globally unique and publicly routable by default).
- Used when you want IPv6 instances to initiate outbound connections but remain unreachable from the internet.

Skill 5.1.2 – Configure Private Networking Connectivity

Private networking connectivity means connecting AWS resources to each other, to other VPCs, or to on-premises networks without traffic traversing the public internet.

VPC Peering

VPC peering creates a direct, private network connection between two VPCs, allowing resources in either VPC to communicate as if they were on the same network.

Source: Connect VPCs using VPC peering

Key characteristics:

- Works across accounts and across AWS Regions (inter-Region peering).
- Traffic stays on the AWS global backbone — it never traverses the public internet.
- **Non-transitive** — if VPC A peers with VPC B, and VPC B peers with VPC C, VPC A cannot reach VPC C through VPC B. Each pair requires its own peering connection.
- CIDR blocks must not overlap.
- Route tables in both VPCs must be updated to route traffic to the peer VPC's CIDR.

When to use VPC peering:

- Small number of VPCs (fewer than ~10) with point-to-point connectivity needs.
 - Cross-account access to shared services.
-

AWS Transit Gateway

AWS Transit Gateway acts as a regional network hub that connects VPCs and on-premises networks through a central gateway, eliminating the need for complex peering meshes.

Key characteristics:

- Supports thousands of VPC attachments.
- **Transitive routing** — VPCs attached to the same Transit Gateway can communicate with each other through it.
- Supports VPN and Direct Connect attachments for hybrid connectivity.
- Route tables on the Transit Gateway control which attachments can communicate.
- Supports **inter-Region peering** between Transit Gateways.
- Supports **multicast** for applications that require one-to-many traffic distribution.

Transit Gateway route tables:

- Each attachment is associated with a route table.
- You can create multiple route tables to segment traffic (e.g., production VPCs cannot reach development VPCs).

```
# Create a Transit Gateway
aws ec2 create-transit-gateway \
  --description "Central hub TGW" \
  --options DefaultRouteTableAssociation=enable,DefaultRouteTablePropagation=enable

# Attach a VPC to the Transit Gateway
aws ec2 create-transit-gateway-vpc-attachment \
  --transit-gateway-id tgw-0abc12345 \
  --vpc-id vpc-0abc12345 \
  --subnet-ids subnet-0abc12345 subnet-0def67890
```

VPC Endpoints

VPC endpoints allow you to privately connect your VPC to supported AWS services without requiring an internet gateway, NAT gateway, VPN, or Direct Connect connection. Traffic between your VPC and the service does not leave the Amazon network.

Source: Connect your VPC to services using AWS PrivateLink

Endpoint types:

Type	Description	Use Case
Gateway endpoint	A gateway that is a target in your route table; free of charge	Amazon S3 and Amazon DynamoDB only
Interface endpoint (PrivateLink)	An elastic network interface (ENI) with a private IP in your subnet; powered by AWS PrivateLink	Most AWS services (EC2, SSM, Secrets Manager, KMS, etc.)
Gateway Load Balancer endpoint	Routes traffic to a fleet of virtual appliances	Third-party security appliances

Interface endpoint (PrivateLink) details:

- Creates an ENI in your subnet with a private IP address.
- DNS resolution for the service endpoint resolves to the private IP when private DNS is enabled.
- Supports endpoint policies to restrict which principals can use the endpoint.
- Billed per hour and per GB of data processed.


```
# Create a gateway endpoint for S3
aws ec2 create-vpc-endpoint \
  --vpc-id vpc-0abc12345 \
  --service-name com.amazonaws.us-east-1.s3 \
  --route-table-ids rtb-0abc12345 \
  --vpc-endpoint-type Gateway

# Create an interface endpoint for SSM
aws ec2 create-vpc-endpoint \
  --vpc-id vpc-0abc12345 \
  --service-name com.amazonaws.us-east-1.ssm \
  --vpc-endpoint-type Interface \
  --subnet-ids subnet-0abc12345 \
  --security-group-ids sg-0abc12345 \
  --private-dns-enabled
```

AWS Site-to-Site VPN

AWS Site-to-Site VPN creates an encrypted IPsec tunnel between your on-premises network and your AWS VPC over the public internet.

Source: What is AWS Site-to-Site VPN?

Key components:

Component	Description
Virtual Private Gateway (VGW)	The AWS-side VPN concentrator; attached to your VPC
Customer Gateway (CGW)	Represents your on-premises VPN device in AWS
VPN connection	The IPsec tunnel between the VGW and CGW; always has two tunnels for redundancy
Transit Gateway VPN attachment	Alternative to VGW; connects VPN to a Transit Gateway for hub-and-spoke topology

Key characteristics:

- Each VPN connection provides two tunnels for high availability.
- Supports static routing or dynamic routing via BGP.
- Maximum throughput per tunnel: ~1.25 Gbps.
- Traffic traverses the public internet (encrypted); latency can vary.

AWS Direct Connect

AWS Direct Connect establishes a dedicated private network connection between your on-premises data center and AWS, bypassing the public internet entirely.

Source: AWS Direct Connect

Connection types:

Type	Bandwidth	Description
Dedicated connection	1 Gbps, 10 Gbps, 100 Gbps	Physical port at a Direct Connect location
Hosted connection	50 Mbps – 10 Gbps	Provisioned by an AWS Direct Connect Partner

Virtual interfaces (VIFs):

- **Private VIF** — connects to a VPC via a Virtual Private Gateway or Direct Connect Gateway.
- **Public VIF** — connects to AWS public services (S3, DynamoDB, etc.) using public IP addresses.
- **Transit VIF** — connects to a Direct Connect Gateway associated with a Transit Gateway.

Direct Connect Gateway:

- Allows a single Direct Connect connection to reach VPCs in multiple AWS Regions.
- Supports up to 10 VGW associations or 3 Transit Gateway associations.

Direct Connect vs. Site-to-Site VPN:

Attribute	Direct Connect	Site-to-Site VPN
Path	Dedicated private circuit	Public internet (encrypted)
Latency	Consistent, low latency	Variable
Bandwidth	Up to 100 Gbps	Up to ~1.25 Gbps per tunnel
Setup time	Weeks (physical provisioning)	Minutes
Cost	Higher (port + data transfer)	Lower
Use case	Production workloads, large data transfers	Backup connectivity, smaller workloads

Skill 5.1.3 – Audit AWS Network Protection Services

AWS provides a layered set of network protection services. A CloudOps engineer must understand how to configure and audit each one.

Amazon Route 53 Resolver DNS Firewall

Route 53 Resolver DNS Firewall filters outbound DNS queries made by resources in your VPC. It allows you to block DNS queries to known malicious domains and to allow only approved domains.

Source: Using DNS Firewall to filter outbound DNS traffic

How it works:

1. You create **rule groups** containing rules that match domain names.
2. Each rule specifies an action: **ALLOW**, **BLOCK** (return NXDOMAIN, NODATA, or a custom response), or **ALERT** (log but allow).
3. Rule groups are associated with VPCs.
4. DNS queries from resources in the VPC are evaluated against the rule groups.

Managed domain lists: AWS provides managed domain lists that are automatically updated with known malicious domains (malware, botnet C&C, phishing). You can use these without maintaining your own lists.

DNS Firewall Advanced: Detects advanced threats such as **Domain Generation Algorithm (DGA)** domains and **DNS tunneling** — techniques used by malware to exfiltrate data or establish command-and-control channels over DNS.

Auditing DNS Firewall:

- Check that rule groups are associated with all VPCs (AWS Firewall Manager can enforce this across an organization).
- Review DNS Firewall logs (published to CloudWatch Logs, S3, or Kinesis Data Firehose) for BLOCK and ALERT actions.
- Use AWS Config rule `vpc-dns-firewall-rule-group-associated` to detect VPCs without DNS Firewall associations.

AWS WAF (Web Application Firewall)

AWS WAF protects web applications from common web exploits and bots that could affect availability, compromise security, or consume excessive resources.

Source: What is AWS WAF?

Supported resources:

- Amazon CloudFront distributions
- Application Load Balancers (ALB)
- Amazon API Gateway REST APIs
- AWS AppSync GraphQL APIs
- Amazon Cognito user pools

Key WAF components:

Component	Description
Web ACL	The top-level container; associated with a protected resource; contains rules and rule groups
Rule	Defines inspection criteria and an action (Allow, Block, Count, CAPTCHA, Challenge)
Rule group	A reusable collection of rules; can be AWS managed, marketplace, or custom
Managed rule groups	Pre-built rule sets maintained by AWS or third parties (e.g., Core Rule Set, Known Bad Inputs, SQL injection, XSS)
IP set	A list of IP addresses or CIDR ranges used in rules
Regex pattern set	Regular expressions used to match request components

WAF capacity units (WCUs): Each rule consumes WCUs; a Web ACL has a default limit of 1,500 WCUs.

Auditing WAF:

- Enable WAF logging to S3, CloudWatch Logs, or Kinesis Data Firehose.
- Review sampled requests in the console to understand what traffic is being blocked or allowed.
- Use AWS Firewall Manager to enforce WAF policies across all accounts in an organization.
- Check that all internet-facing ALBs and CloudFront distributions have a Web ACL associated.

AWS Shield

AWS Shield provides protection against Distributed Denial of Service (DDoS) attacks.

Source: AWS Shield

Shield tiers:

Tier	Description	Cost
Shield Standard	Automatic protection against common, most frequently occurring network and transport layer DDoS attacks; included with all AWS accounts at no extra cost	Free
Shield Advanced	Enhanced DDoS protection for EC2, ELB, CloudFront, Global Accelerator, and Route 53; includes DDoS cost protection, 24/7 access to the AWS Shield Response Team (SRT), and advanced attack diagnostics	\$3,000/month + data transfer fees

Shield Advanced features:

- **Near real-time attack visibility** — detailed metrics and attack diagnostics in CloudWatch.
- **DDoS cost protection** — AWS credits scaling costs incurred during a DDoS attack.
- **AWS Shield Response Team (SRT)** — 24/7 expert support during active attacks.
- **Proactive engagement** — SRT contacts you when your application health degrades during an attack.

- **Health-based detection** — integrates with Route 53 health checks to improve detection accuracy.

Auditing Shield:

- Review the Shield console for active events and attack history.
- Check that critical resources (ALBs, CloudFront, Route 53 hosted zones) are protected under Shield Advanced.
- Use AWS Firewall Manager to enforce Shield Advanced protections across an organization.

AWS Network Firewall

AWS Network Firewall is a managed, stateful network firewall and intrusion detection/prevention service for VPCs. It provides deep packet inspection at the VPC perimeter.

Source: What is AWS Network Firewall?

Key capabilities:

Capability	Description
Stateful inspection	Tracks connection state; allows return traffic automatically
Stateless rules	Fast packet filtering based on 5-tuple (source/dest IP, source/dest port, protocol)
Suricata-compatible IPS rules	Industry-standard rule format for intrusion prevention
Domain list filtering	Allow or block traffic to specific domain names (HTTP/HTTPS)
TLS inspection	Decrypt and inspect TLS traffic (requires certificate configuration)

Deployment architecture:

- Network Firewall is deployed in dedicated **firewall subnets** in each AZ.
- Route tables are updated to route traffic through the firewall endpoints before it reaches its destination.
- Typically deployed in a centralized inspection VPC connected via Transit Gateway.

Auditing Network Firewall:

- Enable firewall logging (alert logs and flow logs) to CloudWatch Logs or S3.
 - Review rule group metrics in CloudWatch for dropped packets and rule matches.
 - Use AWS Firewall Manager to deploy and manage Network Firewall policies across an organization.
 - Check that firewall policies are associated with all firewall instances.
-

Skill 5.1.4 – Optimize the Cost of Network Architectures

Network costs on AWS include data transfer charges, NAT gateway processing fees, VPN connection hours, Direct Connect port fees, and inter-AZ traffic. Optimizing these costs requires understanding traffic patterns and choosing the right connectivity options.

Key cost optimization strategies:

Strategy	Description
Use VPC endpoints for S3 and DynamoDB	Gateway endpoints are free; they eliminate NAT gateway data processing charges for S3/DynamoDB traffic
Deploy NAT gateways per AZ	Avoids cross-AZ data transfer charges; each AZ's private instances use the local NAT gateway
Use interface endpoints for high-volume services	For services like SSM, KMS, or ECR, interface endpoints can be cheaper than routing through a NAT gateway at high volumes
Minimize cross-Region data transfer	Keep workloads in the same Region; use CloudFront to cache content at edge locations instead of transferring data across Regions repeatedly
Use AWS Cost Explorer and VPC Flow Logs	Identify top talkers and unexpected traffic patterns; use flow logs to understand where data transfer costs originate
Right-size Direct Connect	Use hosted connections for lower bandwidth needs instead of dedicated 1G/10G ports
Consolidate VPNs with Transit Gateway	Replace multiple VPN connections with a single Transit Gateway VPN attachment
Use S3 Transfer Acceleration selectively	Only enable it when users are geographically distant from the S3 bucket Region

NAT gateway cost example:

- NAT gateway hourly charge: ~\$0.045/hour per gateway.
 - Data processing: ~\$0.045/GB processed.
 - A single NAT gateway processing 1 TB/month costs approximately \$46 in processing fees alone.
 - Routing S3 traffic through a gateway endpoint instead of NAT gateway eliminates that processing charge entirely.
-

Task 5.2: Configure Domains, DNS Services, and Content Delivery

Skill 5.2.1 – Configure DNS (Route 53 Resolver)

What is Amazon Route 53?

Amazon Route 53 is a highly available and scalable cloud Domain Name System (DNS) web service. It provides three main functions: domain registration, DNS routing, and health checking.

Source: What is Amazon Route 53?

Route 53 Resolver

Route 53 Resolver is the DNS resolver built into every VPC. It answers DNS queries for:

- AWS resources within the VPC (EC2 instances, RDS endpoints, ELB DNS names).
- Private hosted zones associated with the VPC.
- Public DNS names (forwarded to public DNS resolvers).

Source: What is Route 53 VPC Resolver?

Resolver endpoints for hybrid DNS:

For hybrid environments where DNS resolution must span AWS and on-premises networks, Route 53 Resolver supports **inbound** and **outbound** endpoints:

Endpoint Type	Direction	Use Case
Inbound endpoint	On-premises → AWS	On-premises DNS servers forward queries for AWS private hosted zones to the inbound endpoint IP addresses
Outbound endpoint	AWS → On-premises	Route 53 Resolver forwards queries for on-premises domains to on-premises DNS servers via forwarding rules

Resolver rules:

- **Forwarding rules** — forward DNS queries for specific domain names to specified IP addresses (e.g., on-premises DNS servers).
- **System rules** — built-in rules for Route 53 private hosted zones and AWS internal domains; cannot be deleted.
- **Auto-defined rules** — automatically created for reverse DNS lookups and AWS-internal domains.

Rules can be shared across accounts using **AWS Resource Access Manager (RAM)**, allowing a central networking account to manage DNS forwarding rules for the entire organization.

```
# Create an outbound resolver endpoint
aws route53resolver create-resolver-endpoint \
  --creator-request-id "my-outbound-endpoint" \
  --security-group-ids sg-0abc12345 \
  --direction OUTBOUND \
  --ip-addresses SubnetId=subnet-0abc12345 SubnetId=subnet-0def67890

# Create a forwarding rule for on-premises domain
aws route53resolver create-resolver-rule \
  --creator-request-id "corp-forwarding-rule" \
  --rule-type FORWARD \
  --domain-name "corp.example.com" \
  --resolver-endpoint-id rslvr-out-0abc12345 \
  --target-ips Ip=192.168.1.53,Port=53
```

Hosted Zones

A **hosted zone** is a container for DNS records for a domain. Route 53 supports two types:

Type	Description
Public hosted zone	Contains records for a domain accessible from the internet
Private hosted zone	Contains records for a domain accessible only from within associated VPCs

Private hosted zones:

- Must be associated with one or more VPCs.
 - Can be associated with VPCs in other accounts (cross-account association).
 - Override public DNS for the same domain name within the VPC (split-horizon DNS).
 - Require `enableDnsHostnames` and `enableDnsSupport` to be enabled on the VPC.
-

DNS Record Types

Record Type	Description	Example Use
A	Maps a hostname to an IPv4 address	<code>api.example.com → 1.2.3.4</code>
AAAA	Maps a hostname to an IPv6 address	<code>api.example.com → 2001:db8::1</code>
CNAME	Maps a hostname to another hostname	<code>www.example.com → example.com</code> (cannot be used at zone apex)
Alias	Route 53-specific; maps a hostname to an AWS resource; works at zone apex	<code>example.com → my-alb.us-east-1.elb.amazonaws.com</code>
MX	Mail exchange records	Email routing
TXT	Text records	Domain verification, SPF, DKIM
NS	Name server records	Delegates a subdomain to another hosted zone
SOA	Start of authority	Zone metadata
SRV	Service locator	Service discovery

Alias records vs. CNAME records:

- Alias records are free (no charge for DNS queries to AWS resources).
- Alias records support the zone apex (e.g., `example.com`); CNAME records do not.
- Alias records automatically reflect changes to the target resource's IP addresses.

Skill 5.2.2 – Implement Route 53 Routing Policies, Configurations, and Query Logging

Routing Policies

Route 53 supports multiple routing policies that determine how DNS queries are answered.

Source: Choosing a routing policy

Simple Routing

- Returns a single resource for a domain.
- If multiple values are specified, Route 53 returns all values in random order.
- Does not support health checks on individual records.
- Use case: Single resource with no failover requirement.

Weighted Routing

- Routes traffic to multiple resources in proportions you specify.
- Weights are relative (e.g., weight 70 and weight 30 = 70% / 30% split).
- Supports health checks — unhealthy records are excluded from responses.
- Use case: A/B testing, gradual traffic shifting during deployments (canary releases).

```
# Create weighted records for blue/green deployment
aws route53 change-resource-record-sets \
  --hosted-zone-id Z1234567890 \
  --change-batch '{
    "Changes": [{
      "Action": "CREATE",
      "ResourceRecordSet": {
        "Name": "api.example.com",
        "Type": "A",
        "SetIdentifier": "blue",
        "Weight": 90,
        "TTL": 60,
        "ResourceRecords": [{"Value": "1.2.3.4"}]
      }
    }]
  }'
```

Latency-Based Routing

- Routes traffic to the AWS Region that provides the lowest latency for the end user.
- Route 53 uses latency measurements between the user's location and AWS Regions.
- Supports health checks.
- Use case: Multi-Region active-active deployments where you want users routed to the fastest Region.

Failover Routing

- Routes traffic to a primary resource; if the primary is unhealthy, traffic is routed to a secondary (standby) resource.
- Requires health checks on the primary record.
- Use case: Active-passive disaster recovery.

Geolocation Routing

- Routes traffic based on the geographic location of the DNS query originator (continent, country, or US state).
- A default record handles queries from locations not covered by specific geolocation records.
- Use case: Serving region-specific content, complying with data residency requirements, blocking traffic from certain countries.

Source: Geolocation routing

Geoproximity Routing

- Routes traffic based on the geographic location of resources and optionally shifts traffic by specifying a bias value.
 - Requires Route 53 Traffic Flow.
 - Use case: Fine-grained geographic traffic distribution with the ability to expand or shrink the geographic area served by a resource.
-

IP-Based Routing

- Routes traffic based on the IP address of the DNS query originator.
 - You define CIDR blocks and associate them with specific records.
 - Use case: Routing traffic from known corporate IP ranges to internal endpoints, or routing ISP traffic to specific resources.
-

Multivalue Answer Routing

- Returns up to 8 healthy records selected at random.
 - Supports health checks — only healthy records are returned.
 - Not a substitute for a load balancer, but provides basic client-side load balancing.
 - Use case: Simple load distribution across multiple resources.
-

Health Checks

Route 53 health checks monitor the health of your resources and are used by routing policies to exclude unhealthy endpoints from DNS responses.

Health check types:

Type	Description
Endpoint health check	Monitors an IP address or domain name via HTTP, HTTPS, or TCP
Calculated health check	Combines the results of multiple health checks using AND/OR logic
CloudWatch alarm health check	Reports healthy/unhealthy based on the state of a CloudWatch alarm

Health check configuration:

- **Request interval:** 30 seconds (standard) or 10 seconds (fast; higher cost).
- **Failure threshold:** Number of consecutive failures before marking unhealthy (default: 3).
- **String matching:** Optionally check that the response body contains a specific string.
- **Regions:** Route 53 health checkers are distributed globally; you can select which regions perform checks.

Route 53 Query Logging

DNS query logging records information about DNS queries that Route 53 receives, including the domain name queried, the DNS record type, the response code, and the edge location that responded.

Source: Logging and monitoring in Amazon Route 53

Query logging types:

Type	Scope	Destination
Public hosted zone query logging	Queries for public hosted zones	CloudWatch Logs only
Resolver query logging	DNS queries made by resources in a VPC	CloudWatch Logs, S3, or Kinesis Data Firehose

Resolver query log fields include:

- `version`, `account_id`, `region`, `vpc_id`
- `query_timestamp`, `query_name`, `query_type`
- `rcode` (DNS response code: NOERROR, NXDOMAIN, SERVFAIL, etc.)
- `answers` (the DNS response)
- `srcaddr`, `srcport` (source of the query)
- `transport` (UDP or TCP)
- `firewall_rule_action`, `firewall_rule_group_id` (if DNS Firewall is enabled)

Use cases for query logging:

- Security analysis: Detect DNS exfiltration, DGA domains, or unexpected external DNS lookups.
- Troubleshooting: Diagnose DNS resolution failures (NXDOMAIN, SERVFAIL responses).
- Compliance: Audit DNS activity for regulatory requirements.

```
# Enable Resolver query logging for a VPC

aws route53resolver create-resolver-query-log-config \
  --name "vpc-dns-logs" \
  --destination-arn "arn:aws:logs:us-east-1:123456789012:log-group:/aws/route53/resolver"

aws route53resolver associate-resolver-query-log-config \
  --resolver-query-log-config-id rqlc-0abc12345 \
  --resource-id vpc-0abc12345
```

Skill 5.2.3 – Configure Content and Service Distribution

Amazon CloudFront

Amazon CloudFront is a fast content delivery network (CDN) service that securely delivers data, videos, applications, and APIs to customers globally with low latency and high transfer speeds.

Source: What is Amazon CloudFront?

How CloudFront works:

1. You configure a **distribution** with one or more **origins** (S3 buckets, ALBs, EC2 instances, custom HTTP servers).
2. CloudFront caches content at **edge locations** (400+ globally) close to end users.
3. When a user requests content, CloudFront routes the request to the nearest edge location.
4. If the content is cached (cache hit), it is served directly from the edge. If not (cache miss), CloudFront fetches it from the origin, caches it, and serves it.

Distribution types:

- **Standard distribution** — for websites, APIs, and general content delivery.
- **Multi-tenant distribution** — for SaaS providers serving multiple tenants from a single distribution.

CloudFront Origins:

Origin Type	Description
Amazon S3	Static content; use Origin Access Control (OAC) to restrict bucket access to CloudFront only
Application Load Balancer	Dynamic content from web applications
EC2 instance	Custom web servers
API Gateway	REST or HTTP APIs
Custom HTTP origin	Any HTTP/HTTPS server, including on-premises

Origin Access Control (OAC):

- Replaces the older Origin Access Identity (OAI).
- Allows CloudFront to authenticate to S3 using SigV4 signing.
- The S3 bucket policy grants access only to the CloudFront distribution's OAC, preventing direct public access to the bucket.

CloudFront Caching:

Setting	Description
Cache behavior	Rules that define how CloudFront handles requests for specific URL path patterns
Cache policy	Controls what is included in the cache key (headers, query strings, cookies) and the TTL
Origin request policy	Controls what CloudFront includes in requests to the origin (headers, query strings, cookies)
TTL	Minimum, default, and maximum time-to-live for cached objects
Cache invalidation	Removes objects from the cache before TTL expires; charged per invalidation path

Cache key best practices:

- Include only the query strings, headers, and cookies that actually affect the response.
- Avoid including unnecessary headers (e.g., `User-Agent`) in the cache key — this fragments the cache and reduces hit rates.
- Use separate cache behaviors for static assets (long TTL) and dynamic content (short TTL or no caching).

CloudFront Security Features:

Feature	Description
HTTPS	Enforce HTTPS between viewers and CloudFront, and between CloudFront and the origin
AWS WAF integration	Associate a Web ACL with a distribution to filter malicious requests at the edge
AWS Shield	CloudFront distributions are automatically protected by Shield Standard
Geo-restriction	Block or allow requests from specific countries
Signed URLs and signed cookies	Restrict access to content to authenticated users
Field-level encryption	Encrypt specific POST fields at the edge using asymmetric encryption

CloudFront Functions and Lambda@Edge:

Feature	Description	Use Case
CloudFront Functions	Lightweight JavaScript functions that run at edge locations; sub-millisecond execution	URL rewrites, header manipulation, simple request/response transformations
Lambda@Edge	Full Lambda functions that run at Regional edge caches; supports Node.js and Python	Complex authentication, A/B testing, dynamic content generation

CloudFront Pricing:

- Data transfer out to internet (per GB, varies by Region).
- HTTP/HTTPS request charges (per 10,000 requests).
- Invalidation requests (first 1,000 paths/month free; then per path).
- Lambda@Edge and CloudFront Functions invocations.
- **Price classes** allow you to limit which edge locations serve your content to reduce costs (e.g., use only North America and Europe edge locations).

AWS Global Accelerator

AWS Global Accelerator is a networking service that improves the availability and performance of your applications by routing traffic through the AWS global network infrastructure rather than the public internet.

Source: What is AWS Global Accelerator?

How it works:

1. Global Accelerator provides two **static anycast IP addresses** that serve as fixed entry points to your application.
2. Traffic from users enters the AWS network at the nearest **edge location** (AWS Point of Presence).
3. Traffic is then routed over the AWS global backbone to the optimal endpoint (ALB, NLB, EC2 instance, or Elastic IP) in the target Region.

Key benefits:

- **Static IP addresses** — no DNS changes needed when you add or remove endpoints; useful for clients that cache DNS.
- **Improved performance** — traffic travels over the AWS backbone, avoiding congested public internet paths.
- **Automatic failover** — if an endpoint becomes unhealthy, traffic is rerouted to a healthy endpoint within seconds (no DNS TTL wait).
- **Traffic dials** — control the percentage of traffic sent to each endpoint group.
- **Endpoint weights** — distribute traffic across endpoints within an endpoint group.

CloudFront vs. Global Accelerator:

Attribute	CloudFront	Global Accelerator
Primary use	Content caching and delivery	TCP/UDP traffic acceleration
Caching	Yes — caches content at edge locations	No caching
Protocols	HTTP/HTTPS	TCP, UDP (any port)
Static IPs	No (uses DNS)	Yes (two static anycast IPs)
Best for	Static/dynamic web content, APIs	Gaming, IoT, VoIP, non-HTTP workloads, multi-Region failover

Task 5.3: Troubleshoot Network Connectivity Issues

Skill 5.3.1 – Troubleshoot VPC Configurations

Connectivity issues in a VPC almost always come down to one or more of these components being misconfigured: subnets, route tables, NACLs, security groups, NAT gateways, or Transit Gateways.

Systematic Troubleshooting Approach

When an EC2 instance or service cannot communicate, work through the layers:

1. Is the instance running and healthy?
2. Does the subnet have the correct route table?
3. Does the route table have a route to the destination?
4. Does the security group allow the traffic (inbound AND outbound)?
5. Does the NACL allow the traffic (inbound AND outbound, including ephemeral ports)?
6. If using NAT: Is the NAT gateway in a public subnet with an IGW route?
7. If using Transit Gateway: Is the attachment active? Are TGW route tables correct?
8. Is DNS resolving correctly?

Common VPC Connectivity Issues

Issue: Instance in private subnet cannot reach the internet

Check	What to Look For
Route table	Private subnet route table must have <code>0.0.0.0/0 → nat-xxxxxxx</code>
NAT gateway	Must be in a public subnet (not the private subnet)
NAT gateway state	Must be in <code>available</code> state
Public subnet route table	Must have <code>0.0.0.0/0 → igw-xxxxxxx</code>
NAT gateway Elastic IP	Must have an Elastic IP associated
Security group	Outbound rules must allow the traffic
NACL	Both inbound and outbound rules must allow the traffic (including ephemeral return ports 1024–65535)

Issue: Instance cannot be reached from the internet

Check	What to Look For
Public IP	Instance must have a public IP or Elastic IP
Route table	Subnet route table must have <code>0.0.0.0/0 → igw-xxxxxxx</code>
Internet gateway	Must be attached to the VPC
Security group	Inbound rule must allow the traffic on the correct port
NACL	Inbound rule must allow the traffic; outbound rule must allow ephemeral return ports

Issue: Two instances in the same VPC cannot communicate

Check	What to Look For
Security groups	Source instance's SG must be allowed in the destination instance's inbound rules
NACLs	Both subnet NACLs must allow the traffic in both directions
Route tables	Local route (<code>10.0.0.0/16 → local</code>) is always present and cannot be deleted

Transit Gateway Troubleshooting

Issue	Likely Cause	Resolution
VPC attachment not routing	Route not in TGW route table	Add a route to the TGW route table pointing to the VPC attachment
Asymmetric routing	Different route tables for different attachments	Ensure routes are consistent; check propagation settings
Overlapping CIDRs	Two VPCs with the same CIDR attached	VPCs with overlapping CIDRs cannot communicate via TGW
Attachment in <code>pending</code> state	Subnet not in the correct AZ	Ensure subnets are in AZs where TGW is available

NAT Gateway Troubleshooting

Issue	Likely Cause	Resolution
<code>ErrorPortAllocation</code>	NAT gateway has exhausted its 55,000 connection limit per Elastic IP	Add additional Elastic IPs to the NAT gateway, or distribute traffic across multiple NAT gateways
High latency	Cross-AZ NAT traffic	Deploy a NAT gateway in each AZ; update route tables to use the local AZ's NAT gateway
NAT gateway in <code>failed</code> state	Underlying infrastructure issue	Delete and recreate the NAT gateway

AWS Reachability Analyzer

AWS Reachability Analyzer is a network diagnostics tool that analyzes the network path between a source and destination in your VPC and identifies the configuration component that is blocking connectivity.

- Supports sources and destinations: EC2 instances, ENIs, internet gateways, VPC endpoints, VPN gateways, Transit Gateway attachments.
- Does not send actual traffic — it analyzes the configuration.
- Returns either "reachable" (with the path) or "not reachable" (with the blocking component).

```
# Create a reachability analysis
aws ec2 create-network-insights-path \
  --source eni-0abc12345 \
  --destination eni-0def67890 \
  --protocol TCP \
  --destination-port 443

aws ec2 start-network-insights-analysis \
  --network-insights-path-id nip-0abc12345
```

Skill 5.3.2 – Collect and Interpret Networking Logs

VPC Flow Logs

VPC Flow Logs capture information about the IP traffic going to and from network interfaces in your VPC. They are the primary tool for network traffic analysis and troubleshooting.

Source: Logging IP traffic using VPC Flow Logs

Flow log scope:

- **VPC level** — captures all traffic in the VPC.
- **Subnet level** — captures all traffic in a specific subnet.
- **ENI level** — captures traffic for a specific network interface.

Flow log destinations:

- **Amazon CloudWatch Logs** — enables real-time analysis with metric filters and CloudWatch Insights queries.
- **Amazon S3** — enables long-term storage and analysis with Amazon Athena.
- **Amazon Kinesis Data Firehose** — enables streaming to third-party SIEM tools.

Default flow log record fields:

Field	Description
<code>version</code>	Flow log version
<code>account-id</code>	AWS account ID
<code>interface-id</code>	ENI ID
<code>srcaddr</code>	Source IP address
<code>dstaddr</code>	Destination IP address
<code>srcport</code>	Source port
<code>dstport</code>	Destination port
<code>protocol</code>	IANA protocol number (6=TCP, 17=UDP, 1=ICMP)
<code>packets</code>	Number of packets transferred
<code>bytes</code>	Number of bytes transferred
<code>start</code> / <code>end</code>	Start and end time of the flow
<code>action</code>	<code>ACCEPT</code> or <code>REJECT</code>
<code>log-status</code>	<code>OK</code> , <code>NODATA</code> , or <code>SKIPDATA</code>

Custom flow log fields (v5+): Additional fields include `vpc-id`, `subnet-id`, `instance-id`, `tcp-f` `lags`, `traffic-path`, `flow-direction`, and more.

Interpreting flow logs:

```
# REJECT entries indicate traffic blocked by security groups or NACLs
2 123456789012 eni-0abc12345 10.0.1.5 10.0.2.10 54321 443 6 10 840 1620000000 1620000
060 REJECT OK

# ACCEPT entries show allowed traffic
2 123456789012 eni-0abc12345 10.0.1.5 10.0.2.10 54321 443 6 10 840 1620000000 1620000
060 ACCEPT OK
```

Querying flow logs with CloudWatch Insights:

```
# Find all rejected traffic to port 443
fields @timestamp, srcAddr, dstAddr, srcPort, dstPort, action
| filter action = "REJECT" and dstPort = 443
| sort @timestamp desc
| limit 100
```

Querying flow logs with Athena:

```
-- Find top source IPs by bytes transferred
SELECT srcaddr, SUM(bytes) AS total_bytes
FROM vpc_flow_logs
WHERE action = 'ACCEPT'
GROUP BY srcaddr
ORDER BY total_bytes DESC
LIMIT 20;
```

Elastic Load Balancing Access Logs

ELB access logs capture detailed information about requests sent to your load balancer, including the time, client IP, latencies, request paths, and server responses.

Key fields in ALB access logs:

Field	Description
<code>type</code>	Request type (http, https, h2, grpc, ws, wss)
<code>time</code>	Timestamp of the request
<code>elb</code>	Load balancer name
<code>client:port</code>	Client IP and port
<code>target:port</code>	Target IP and port
<code>request_processing_time</code>	Time from receiving request to sending it to the target
<code>target_processing_time</code>	Time from sending request to target to receiving response
<code>response_processing_time</code>	Time from receiving target response to sending response to client
<code>elb_status_code</code>	HTTP status code from the load balancer
<code>target_status_code</code>	HTTP status code from the target
<code>request</code>	HTTP method, URL, and HTTP version
<code>user_agent</code>	Client user agent string
<code>ssl_cipher</code> / <code>ssl_protocol</code>	TLS details

Troubleshooting with ELB logs:

- `504 Gateway Timeout` — target did not respond in time; check target health and application performance.
- `502 Bad Gateway` — target returned an invalid response; check application logs.
- `000` target status code — load balancer could not connect to the target; check security groups and target health.
- High `target_processing_time` — application is slow; investigate with APM tools.

AWS WAF Web ACL Logs

WAF logs record every request evaluated by a Web ACL, including the action taken (ALLOW, BLOCK, COUNT, CAPTCHA) and which rule matched.

Key log fields:

- `timestamp`, `formatVersion`
- `webaclId` — the Web ACL ARN
- `action` — ALLOW, BLOCK, COUNT, CAPTCHA, CHALLENGE
- `terminatingRuleId` — the rule that terminated the evaluation
- `terminatingRuleType` — REGULAR, RATE_BASED, GROUP
- `httpRequest` — full request details (URI, headers, body)
- `ruleGroupList` — results from each rule group evaluated

Use cases:

- Identify which rule is blocking legitimate traffic (false positives).
 - Analyze attack patterns to tune WAF rules.
 - Audit compliance with security policies.
-

Amazon CloudFront Logs

CloudFront standard logs (access logs) record every viewer request received by CloudFront, including the date, time, edge location, bytes served, HTTP status code, and cache hit/miss status.

Key log fields:

Field	Description
<code>date</code> / <code>time</code>	Request timestamp
<code>x-edge-location</code>	Edge location that served the request
<code>sc-bytes</code>	Bytes sent to the viewer
<code>c-ip</code>	Viewer IP address
<code>cs-method</code>	HTTP method
<code>cs-uri-stem</code>	Request URI path
<code>sc-status</code>	HTTP status code
<code>x-edge-result-type</code>	<code>Hit</code> , <code>Miss</code> , <code>Error</code> , <code>LimitExceeded</code>
<code>x-edge-request-id</code>	Unique request ID
<code>x-host-header</code>	Host header value
<code>cs-protocol</code>	<code>https</code> or <code>http</code>
<code>time-taken</code>	Total time to serve the request
<code>ssl-protocol</code> / <code>ssl-cipher</code>	TLS details

CloudFront Real-Time Logs:

- Deliver log records to Kinesis Data Streams within seconds of the request.
- Configurable sampling rate (1%–100%).
- Useful for real-time dashboards and security monitoring.

Skill 5.3.3 – Identify and Remediate CloudFront Caching Issues

Caching issues in CloudFront manifest as either too much caching (stale content served to users) or too little caching (high origin load, poor performance).

Diagnosing Cache Behavior

Cache hit ratio is the percentage of requests served from the CloudFront cache without going to the origin. A low cache hit ratio means most requests are going to the origin, which increases latency and origin load.

Check the cache hit ratio:

- In the CloudFront console: Monitoring → Cache statistics → Cache hit rate.
- In CloudFront access logs: `x-edge-result-type` field — `Hit` vs. `Miss`.
- In CloudWatch: `CacheHitRate` metric per distribution.

Common causes of low cache hit ratio:

Cause	Description	Remediation
Too many cache key variations	Cache policy includes unnecessary headers, cookies, or query strings	Remove unused variables from the cache key; use separate cache behaviors for dynamic vs. static content
Short TTL	Objects expire quickly and are frequently re-fetched from origin	Increase <code>Cache-Control: max-age</code> on origin responses, or set a higher default TTL in the cache policy
No-cache headers from origin	Origin sends <code>Cache-Control: no-cache</code> or <code>no-store</code>	Override origin cache headers using a cache policy with a minimum TTL > 0
Query string variations	Different query string values create different cache entries	Normalize query strings; only forward query strings that affect the response
Personalized content cached	User-specific content (e.g., session tokens in query strings) is being cached	Exclude personalized content from caching; use signed URLs for private content

Invalidating Cached Content

When you update content at the origin, CloudFront continues to serve the cached version until the TTL expires. To force CloudFront to fetch fresh content immediately, create an **invalidation**.

```
# Invalidate specific files
aws cloudfront create-invalidation \
  --distribution-id E1234567890 \
  --paths "/images/logo.png" "/css/styles.css"

# Invalidate all files (wildcard)
aws cloudfront create-invalidation \
  --distribution-id E1234567890 \
  --paths "/*"
```

Invalidation considerations:

- First 1,000 invalidation paths per month are free; additional paths are charged.
- Invalidations propagate to all edge locations (typically within a few minutes).
- For frequent updates, use **versioned file names** (e.g., `styles.v2.css`) instead of invalidations — this is more efficient and avoids charges.

CloudFront Error Responses

HTTP Status	Meaning	Common Cause
403 Forbidden	Access denied	S3 bucket policy does not allow CloudFront OAC; WAF blocking the request; geo-restriction
404 Not Found	Object not found at origin	Object does not exist in S3; incorrect origin path
502 Bad Gateway	CloudFront could not connect to origin	Origin is down; security group blocks CloudFront IPs; SSL certificate mismatch
503 Service Unavailable	Origin returned 503	Origin is overloaded or throttling requests
504 Gateway Timeout	Origin did not respond in time	Origin is slow; connection timeout too short

Custom error pages: Configure CloudFront to serve a custom error page (e.g., a branded 404 page) instead of the default CloudFront error response. You can also configure a minimum TTL for error responses to prevent CloudFront from repeatedly hitting an unavailable origin.

Skill 5.3.4 – Identify and Troubleshoot Hybrid Connectivity and Private Connectivity Issues

Hybrid Connectivity Troubleshooting (VPN and Direct Connect)

Site-to-Site VPN issues:

Issue	Likely Cause	Resolution
Tunnel is down	IKE/IPsec negotiation failure	Verify pre-shared key, IKE version, encryption algorithms match on both sides
Tunnel is up but no traffic	Routing issue	Check BGP session (for dynamic routing) or static routes; verify security groups and NACLs
One tunnel is down	Normal — one tunnel may be in standby	Both tunnels should be configured; AWS performs maintenance on tunnels periodically
High latency	Traffic routing over public internet	Consider Direct Connect for consistent latency

Monitoring VPN tunnels:

- CloudWatch metrics: `TunnelState` (1=up, 0=down), `TunnelDataIn`, `TunnelDataOut`.
- Set a CloudWatch alarm on `TunnelState` to alert when a tunnel goes down.

Direct Connect issues:

Issue	Likely Cause	Resolution
BGP session down	BGP configuration mismatch	Verify ASN, BGP peer IPs, and MD5 authentication key
No routes advertised	BGP not advertising prefixes	Check route filters and BGP community tags
Asymmetric routing	Traffic going out via Direct Connect but returning via internet	Ensure consistent routing on both sides; use BGP communities to prefer Direct Connect
Physical layer issue	Fiber or hardware problem	Contact AWS Support or the Direct Connect partner

Private Connectivity Troubleshooting (VPC Endpoints and PrivateLink)

Interface endpoint issues:

Issue	Likely Cause	Resolution
DNS not resolving to private IP	Private DNS not enabled on endpoint	Enable private DNS on the interface endpoint
<code>AccessDenied</code> when using endpoint	Endpoint policy is too restrictive	Review and update the VPC endpoint policy
Cannot reach endpoint	Security group on endpoint ENI blocks traffic	Add an inbound rule to the endpoint's security group allowing traffic from the source
<code>enableDnsSupport</code> not enabled	VPC DNS support disabled	Enable <code>enableDnsSupport</code> on the VPC

Gateway endpoint issues (S3/DynamoDB):

Issue	Likely Cause	Resolution
Traffic still going through NAT	Route table not updated	Ensure the gateway endpoint is added to the correct route table
<code>AccessDenied</code>	Endpoint policy or S3 bucket policy is blocking	Review endpoint policy; check S3 bucket policy for <code>aws:sourceVpce</code> conditions

Skill 5.3.5 – Configure and Analyze Amazon CloudWatch Network Monitoring Services

CloudWatch provides several network-specific monitoring capabilities that help CloudOps engineers gain visibility into network performance and troubleshoot issues.

CloudWatch Network Monitor

Amazon CloudWatch Network Monitor provides active network monitoring between your AWS resources and on-premises networks. It measures latency, packet loss, and jitter using probes deployed in your VPCs.

Key concepts:

- **Monitor** — a top-level resource that groups probes.
- **Probe** — an active monitoring agent deployed in a VPC subnet that sends ICMP or TCP packets to a destination.
- **Metrics** — `RTT` (round-trip time), `PacketLoss`, `Jitter` published to CloudWatch.

Use cases:

- Monitor the health of VPN tunnels and Direct Connect connections.
 - Detect network degradation before it impacts applications.
 - Compare performance of VPN vs. Direct Connect paths.
-

CloudWatch Network Flow Monitor

Amazon CloudWatch Network Flow Monitor provides visibility into network flows between workloads in your VPCs, helping you understand traffic patterns and identify performance issues.

Key capabilities:

- Monitors TCP flows between EC2 instances, containers, and other resources.
 - Provides metrics on retransmissions, round-trip time, and data volume per flow.
 - Identifies top talkers and unusual traffic patterns.
-

VPC Flow Logs with CloudWatch Metrics

You can create **CloudWatch metric filters** on VPC flow log groups to generate metrics and alarms from flow log data:

```
# Create a metric filter to count rejected traffic
aws logs put-metric-filter \
  --log-group-name /aws/vpc/flowlogs \
  --filter-name RejectedConnections \
  --filter-pattern '[version, account, eni, source, destination, srcport, destport, p
rotocol, packets, bytes, windowstart, windowend, action="REJECT", flowlogstatus]' \
  --metric-transformations \
    metricName=RejectedConnections,metricNamespace=VPCFlowLogs,metricValue=1

# Create an alarm on the metric
aws cloudwatch put-metric-alarm \
  --alarm-name HighRejectedConnections \
  --metric-name RejectedConnections \
  --namespace VPCFlowLogs \
  --statistic Sum \
  --period 300 \
  --threshold 100 \
  --comparison-operator GreaterThanThreshold \
  --evaluation-periods 1 \
  --alarm-actions arn:aws:sns:us-east-1:123456789012:NetworkAlerts
```

CloudWatch Internet Monitor

Amazon CloudWatch Internet Monitor provides visibility into how internet issues affect the performance and availability of your application for end users. It monitors the health of the internet between your application and your users' locations.

Key capabilities:

- Identifies internet routing issues, congestion, and outages affecting specific geographic areas.
- Provides health scores for traffic from different cities and countries.
- Publishes health events to EventBridge for automated responses.
- Integrates with Route 53 ARC (Application Recovery Controller) for automated traffic shifting.

Use cases:

- Detect when users in a specific region are experiencing degraded performance due to an internet issue.
- Automatically shift traffic to a different Region or CloudFront origin when internet health degrades.
- Understand the geographic distribution of your users and their connectivity quality.

```
# Create an Internet Monitor monitor
aws internetmonitor create-monitor \
  --monitor-name my-app-monitor \
  --resources \
    "arn:aws:cloudfront::123456789012:distribution/E1234567890" \
    "arn:aws:ec2:us-east-1:123456789012:vpc/vpc-0abc12345" \
  --traffic-percentage-to-monitor 100 \
  --max-city-networks-to-monitor 500
```

Summary Reference Table

Service / Feature	Primary Purpose	Key Exam Points
VPC	Isolated virtual network	Subnets, route tables, IGW, NAT GW, NACLs, security groups
Security Groups	Instance-level firewall	Stateful; allow-only; can reference other SGs
NACLs	Subnet-level firewall	Stateless; allow and deny; rules evaluated in order
NAT Gateway	Outbound internet for private subnets	Deploy one per AZ; requires Elastic IP; managed by AWS
VPC Endpoints	Private access to AWS services	Gateway (S3/DynamoDB, free); Interface (PrivateLink, charged)
Transit Gateway	Hub-and-spoke VPC connectivity	Transitive routing; supports VPN and Direct Connect
Site-to-Site VPN	Encrypted on-premises connectivity	Two tunnels; up to ~1.25 Gbps; variable latency
Direct Connect	Dedicated on-premises connectivity	Consistent latency; up to 100 Gbps; weeks to provision
Route 53 Resolver	VPC DNS resolution	Inbound/outbound endpoints for hybrid DNS
Route 53 Routing Policies	Traffic management	Simple, Weighted, Latency, Failover, Geolocation, IP-based, Multivalue
Route 53 Health Checks	Endpoint monitoring	Used by routing policies to exclude unhealthy endpoints
DNS Firewall	Outbound DNS filtering	Blocks malicious domains; detects DGA and DNS tunneling
AWS WAF	Web application firewall	Protects ALB, CloudFront, API Gateway; managed rule groups
AWS Shield	DDoS protection	Standard (free, automatic); Advanced (\$3K/month, SRT access)
AWS Network Firewall	VPC perimeter firewall	Stateful inspection; Suricata IPS rules; domain filtering
CloudFront	Content delivery network	Edge caching; OAC for S3; WAF integration; signed URLs
Global Accelerator	TCP/UDP traffic acceleration	Static anycast IPs; AWS backbone routing; fast failover
VPC Flow Logs	Network traffic logging	ACCEPT/REJECT records; CloudWatch Logs or S3
Reachability Analyzer	Connectivity diagnostics	Analyzes config without sending traffic
CloudWatch Network Monitor	Active network monitoring	Latency, packet loss, jitter for hybrid connectivity

Service / Feature	Primary Purpose	Key Exam Points
CloudWatch Internet Monitor	Internet health monitoring	Geographic performance visibility; EventBridge integration

Sources:

- *Amazon VPC User Guide*
- *Amazon Route 53 Developer Guide*
- *Amazon CloudFront Developer Guide*
- *AWS Global Accelerator Developer Guide*
- *AWS Network Firewall Developer Guide*
- *AWS WAF Developer Guide*
- *AWS Site-to-Site VPN User Guide*
- *AWS Direct Connect User Guide*
- *AWS Certified SysOps Administrator – Associate Exam Guide (SOA-Co3)*